Rubric-Scoped Grading of Handwritten Answer Scripts with Course-Filtered Retrieval and a Dual-Engine Language Model Pipeline

Harithe Induwara Bandara Wickramarathne
Faculty of Computing
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
haritheb@gmail.com

Akila Manujith Senarathna
Faculty of Computing
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
akilamanujith@gmail.com

B. K. H. M. Buddhila Lakshan Herath
Faculty of Computing
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
Buddhilaherath28@gmail.com

L. Supun Buddhika Hemarathne
Faculty of Computing
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
Liyanagesupun10@gmail.com

***Abstract*—Handwritten examinations remain the dominant summative assessment format in many universities, yet marking them is slow and hard to keep consistent across a large cohort. This paper presents the handwritten grading subsystem of a modular assessment platform, and the engineering contract that lets a language model mark a scanned script without silently producing an unjustified mark. The pipeline conditions each page, recognises the handwriting through a staged retry policy, segments the noisy transcript into per-question answers with a marker grammar that tolerates recognition errors, retrieves lecture context restricted to the course being graded, and marks every rubric question through an independent call to a low-rank adapted eight billion parameter model behind a health-gated router with an instruction-tuned fallback engine. Component experiments on synthetic data show that the segmenter attributes every answer to the correct question under four of six degradation conditions, where a digit-only baseline reaches at most seventy one per cent and fails completely on three; that twelve of fourteen malformed grading responses are recovered and the other two rejected rather than scored, with no case yielding a silently incorrect mark; and that course-filtered retrieval rejects all fifty four out-of-scope queries with no foreign material entering a grading prompt. Agreement with human markers on authentic scripts is not yet measured and is the subject of an ethics approved follow-up study.**

Keywords—automated short answer grading, handwriting recognition, retrieval augmented generation, large language models, educational assessment, low-rank adaptation

I. Introduction

Written examinations remain the primary instrument of summative assessment in most degree programmes, and a single second-year module can produce several hundred handwritten scripts that must be marked in a short release window. The cost is not only the hours spent reading; it is also the drift that appears when one rubric is applied by different markers, or by the same marker late in a long session, and the delay before a student receives usable feedback. Automating part of this work has been an active research goal for two decades [1], [2].

Automatic short answer grading has moved from hand-engineered similarity features through corpus-based semantics to representation learning and, recently, to large language models [1], [3]. The evidence for language models is mixed: a pre-trained model without task-specific training performs comparably to older hand-engineered systems but worse than specialised models [4], and a controlled comparison of zero-shot, few-shot and fine-tuned settings found none dependable enough for operational use [5]. Two properties of that literature matter here. First, it almost always assumes the answer arrives as clean digital text, already separated from the rest of the script, so recognition and segmentation lie outside the evaluated system. Second, it measures agreement with a reference score and rarely reports the operational behaviour a marking system must guarantee: what happens when the model endpoint fails mid-batch, how a mark is traced to the evidence that produced it, and how the system establishes that a script belongs to the student it is credited to.

This paper addresses that combined gap through the handwritten grading subsystem of Gradex, a modular assessment platform built by the authors, which turns photographed scripts into per-question marks, per-criterion breakdowns and feedback aligned to a lecturer-supplied rubric. We contribute a conditioning and recognition cascade tuned for ballpoint handwriting rather than print; a marker grammar that segments a noisy transcript into per-question answers, with a conservative policy that scores an unmatched question zero rather than offering the whole script as its answer; retrieval hard-scoped to the module being graded and queried from the rubric rather than the student answer; one model call per question through a health-gated dual-engine router with a normalisation layer that recovers marks from malformed output or refuses it, but never fabricates one; and reproducible experiments on segmentation robustness, response recovery and retrieval isolation. Scope is explicit: Section IV measures the deterministic components and the failure behaviour, not agreement with human marks, because the data used so far is synthetic and a study on real student work needs institutional ethical clearance that is being sought.

II. Related Work

Burrows et al. organise two decades of short answer grading into five eras and show that the field converged on an era of evaluation built on shared corpora [1]; Haller et al. continue that history from word embeddings to transformer encoders [3], and Ramesh and Sanampudi report that reliability and content relevance remain the weakest dimensions of automated scoring for longer responses [2].

Work on generative graders is cautious: Kortemeyer found performance comparable to hand-engineered systems but below specialised models, with the advantage of needing no task-specific training [4], and Chamieh et al. reported a fine-tuned model approaching but not reaching the supervised upper bound [5]. Using a language model to judge free text shows position and verbosity biases [6], one reason our system constrains each call to a single question and an explicit criterion list. On the recognition side, work runs from the IAM database that made comparable evaluation possible [7] to transformer encoder-decoder recognisers that set the state of the art [8], with open-source pipelines such as Tesseract still widely deployed [9]; the operators in Section III-B are classical, namely Otsu's threshold selection [10], the Hough transform [11], the Canny detector [12] and non-local means denoising [13], applied through OpenCV [14].

Retrieval-augmented generation pairs a parametric model with an index so output can be traced to retrieved evidence [15], and the design space is surveyed in detail [16]; we use sentence embeddings of the kind introduced by Sentence-BERT [17] in a distilled form [18], because grading queries are short and retrieval runs inside a request. Low-rank adaptation specialises a large decoder on one consumer accelerator by freezing base weights and training small rank decomposition matrices [19], and quantised adaptation extends this to eight billion parameter models within the memory of a free cloud notebook [20]; our base model belongs to the Llama 3 family [21]. Output stability is a functional requirement for marking rather than a nicety, and Atil et al. showed that served models do not reproduce identical output across identical runs even at temperature zero with a fixed seed [22]. That result shaped two decisions below: greedy decoding at both engines, and a deterministic layer that recomputes totals from per-question marks. Reviews of artificial intelligence in higher education note that assessment applications are deployed faster than their ethical implications are examined [23], and model-produced scores are cautioned against as a replacement for a marker [24].

III. System Design

A. Overview and Rubric Acquisition

The subsystem is a FastAPI service mounted as a sub-application inside a shared platform server, sharing authentication and course data with the other assessment modules while keeping its own database, vector store and configuration. Rubrics, rosters, submissions and results are held in MongoDB, lecture material as embedded chunks in a persistent Chroma index [25], and a React client drives the workflow and polls for progress. Table I lists the deployed configuration, in which every value is a constant or environment default in the released code. A run is defined by a rubric rather than a reference answer: the uploaded marking scheme is parsed into per-question number, question text, ordered criterion list and available marks; criteria with no individual marks share the question total evenly; and question numbers are canonicalised to zero-padded two-digit strings so a scheme written with Roman or unpadded labels still joins with the segmenter output. An unavailable parser is surfaced as a distinct error rather than an empty rubric, which would be indistinguishable from a paper with no questions.

TABLE I. DEPLOYED CONFIGURATION OF THE GRADING PIPELINE

Stage	Parameter	Value
Conditioning	Downscale threshold, long edge	2200 px
Conditioning	Skew range, minimum correction	30 deg, 0.3 deg
Recognition	Primary engine, escalation	3, then 3, 2, 2
Recognition	Usable page threshold	8 alphanumeric chars
Segmentation	Question labels resolved	1 to 20, Arabic or Roman
Retrieval	Embedding model	all-MiniLM-L6-v2
Retrieval	Chunk unit, minimum length	page or slide, 40 chars
Retrieval	Top-k, ceiling, margin	3, 0.45, 0.12
Grading	Model calls per script	one per rubric question
Grading	Read, connect timeout	120 s, 20 s
Grading	Retries, backoff	3, 2 s per attempt
Grading	Health verdict cache	60 s
Grading	Decoding	greedy, temperature 0
Orchestration	Concurrent pages per script	3
Orchestration	Client poll interval	2 s

B. Page Conditioning and Recognition

Each page is reduced so its long edge does not exceed 2200 pixels, converted to greyscale, deskewed and denoised. Deskew estimates the text angle twice: the primary estimate binarises with Otsu's method [10] and fits a minimum-area rectangle to the foreground, trusted only when at least 200 foreground pixels are present and the angle lies between 0.3 and 30 degrees, and the fallback detects segments with the Canny detector and a probabilistic Hough transform [11], [12], keeps those within 30 degrees of horizontal and takes the median. Rotations below 0.3 degrees are skipped so repeated resampling does not blur strokes for no gain, and denoising uses non-local means [13] at a mild strength. What the stage omits matters as much: adaptive thresholding followed by erosion, standard for printed documents, was removed after it broke thin ballpoint strokes into fragments the recogniser then dropped, so the conditioned page stays greyscale and unbinarised and the original is retained as an alternative input.

Recognition is an escalation rather than a single call. The conditioned page goes first to the neural engine of a commercial service [26]; if the result is unusable, the resized original is sent to the same engine, then to the alternative engine, and finally the conditioned page to the alternative engine. A result is usable only if it is non-empty, carries no error prefix and holds at least eight alphanumeric characters, which excludes a stray punctuation mark returned for a page the service could not read. Pages surviving no attempt are recorded as explicitly empty rather than omitted, so a marker can see that a page existed and failed. Document-format scripts are rasterised at twice nominal resolution into the same path, and the pages of one script are recognised three at a time so network latency overlaps without exhausting the service quota. Two transcripts are stored per script: a raw one with page banners and empty-page notices intact for audit, and a cleaned one, which the segmenter consumes.

C. Segmenting the Transcript into Answers

A recognised script is one long string and a rubric is a list of questions; joining them decides whether a mark is attributable at all. The segmenter searches the cleaned transcript for line-initial markers under a grammar accepting the word form, the abbreviated form with optional punctuation and the bare enumerated form, and accepting as a label an Arabic number, a Roman numeral up to twenty, or one of the single characters that recognition commonly substitutes for a handwritten one. A negative lookahead rejects a label immediately followed by a letter, preventing an ordinary sentence beginning with the letter Q from being read as a header. Only the first occurrence of each label is kept, each answer runs to the next accepted label, trailing artefacts are trimmed, and labels absent from the rubric are discarded.

The resolution policy is conservative and is recorded with every mark. A question whose own segment was found is marked from it and labelled split; a script with no marker anywhere is treated as a single-answer paper and the cleaned transcript is presented once, labelled full; a question with no segment in a script where other markers were found is scored zero and labelled empty. The third case is the important one, because offering the whole script as the answer to a question that was never attempted would let content written for another question earn credit and no later check could detect it. The pipeline therefore accepts a false zero a marker can see and override instead of a false positive that is invisible.

D. Course-Scoped Retrieval

Lecture slides and notes are indexed one vector per page or slide, tagged with course, source file and page number. The grading query is built from the rubric, concatenating the topic, up to three question texts and up to two criteria from each, falling back to the student answer only when the rubric yields no text: a query built from a wrong answer retrieves material matching the misconception, which is the context least useful for judging that answer against the scheme. Retrieval applies a hard metadata filter on the course, so material from another module cannot be returned even when semantically closer, and no cross-course fallback exists. Three times the required candidates are fetched and then filtered: chunks under forty characters are dropped as slide furniture, chunks beyond a distance of 0.45 are dropped as irrelevant, and of the rest only those within 0.12 of the closest hit are kept before truncation to the top three. When nothing survives, the prompt is instructed to mark from the rubric and answer alone rather than being given prose about missing material that the model might treat as content. Whether context was used, how many chunks and their distances are stored per question beside the mark.

E. Rubric-Scoped Grading and Engine Routing

Grading is issued one question at a time, each request carrying the question identity, a rubric array containing only that question, the retrieved context and the segmented answer. Marking questions independently keeps prompts short, prevents the model from redistributing marks between questions to make a total look reasonable, makes a single question re-gradable without disturbing its neighbours, and yields per-question latency the client turns into progress; the cost is more model calls per script, which the orchestration layer absorbs. The router prefers the fine-tuned model and falls back to a hosted instruction-tuned model. Before a batch the primary endpoint is probed once, treating any response other than a not-found status or a server error as evidence that it is alive, and the verdict is cached for sixty seconds so a batch of hundreds of questions does not re-probe per call. During grading a server error or rate-limit response is retried up to three times with a linearly increasing delay; a not-found, forbidden or gone response marks the endpoint unavailable at once, because those statuses indicate a stale tunnel address rather than a transient fault; any other client error fails fast. Once the primary is unavailable the remaining questions skip it instead of each paying the retry budget again. The engine behind each mark is stored with it, and a script graded partly by each engine is labelled mixed rather than reported under one engine's name.

Model output is not trusted as returned. A normalisation layer strips code fences, unwraps the envelope objects a notebook-hosted server adds around generated text, extracts a grading object embedded in echoed prompt text, coerces numeric fields supplied as strings, and reconciles question identifiers written with or without zero padding. The reported total is then recomputed as the sum of per-question marks rather than read from the model, so an internally inconsistent total cannot enter the record. If no mark can be located the response is rejected and the router retries or fails over; a stub response is emitted only when no fallback engine is configured at all, and it is labelled so it cannot be mistaken for a grade.

F. The Fine-Tuned Grading Model

The primary engine is an eight billion parameter Llama 3.1 base model [21] loaded in four-bit quantised form and specialised by low-rank adaptation [19], [20], [27] with rank sixteen and scaling sixteen, no adapter dropout, adapters on the four attention and three feed-forward projections, gradient checkpointing enabled and a maximum sequence length of 2048 tokens. Training and inference run on a single cloud notebook accelerator; adapters are stored separately from the base weights and re-attached at load time, which keeps the artefact small enough to version but requires the serving stack to load the base model first. The model is served behind a small asynchronous web application exposed through a tunnel, with a prompt of four fixed fields naming the topic, the rubric, the reference context and the student answer. Decoding is greedy at both engines and the fallback is additionally constrained to emit a JSON object, because a marking system must return the same mark for the same evidence; greedy decoding is necessary but not sufficient for that [22], which is why the normalisation layer recomputes totals.

The supervised data used so far is synthetic and instructor-authored, arranged so each rubric item is paired with answers spanning the whole mark range rather than only the correct and incorrect extremes. Table II sets out the four response classes. The partial class exists to stop the model learning a length or fluency heuristic, and the invalid class to separate an answer that is wrong from a page that contains no answer, because those cases carry different consequences for a student.

TABLE II. RESPONSE CLASSES IN THE SYNTHETIC GRADING CORPUS

Class	Definition	Expected marking behaviour
Golden	Every criterion addressed with correct content and terminology	Full or near-full marks, all criteria satisfied
Partial	Some criteria correct, others omitted or under-developed	Intermediate mark traceable to satisfied criteria
False	Fluent and confident but factually incorrect for the topic	Low mark despite surface quality, error named in feedback
Invalid	Off-topic text, unattempted question, or unreadable page	Zero, reason distinguished from a wrong answer

G. Identity Assurance, Orchestration and Progress

A mark is worthless if attached to the wrong student, so grading is gated on identity before any model call. Roster identifiers are read from an uploaded spreadsheet and normalised by removing separators and folding case. A script's identifier comes from its containing folder name when that name matches the institutional pattern, and only otherwise is the first page recognised and searched for a labelled or bare identifier, which keeps the check cheap; an operator may override any identifier, and precedence is manual, then folder, then recognised. Validation classifies every pairing as matched, missing paper, extra paper, duplicate paper, duplicate roster entry or unreadable identifier. The last three are hard blockers that refuse the batch, since each means a mark could be written to the wrong record; the first two are soft warnings a lecturer may knowingly accept, for instance when a registered student was absent, and accepting them requires an explicit opt-in rather than being the default. Only matched scripts are graded.

A validated batch runs as a background task with scripts in sequence and the pages of a script concurrent: sequential scripts bound peak memory and keep the engines within rate limits, while concurrency within a script hides recognition latency where it dominates. A progress document per script records the stage, pages and questions completed against their totals, the question being marked and timestamps; a script yielding no text is marked skipped with a reason rather than failing the batch, and an exception marks that script failed and lets the batch continue. The client polls every two seconds and reports completion as the proportion of scripts in a terminal state, counting graded, failed and skipped alike. Time estimates are running averages over work already done in the current stage, dividing elapsed time by units completed and multiplying by units remaining, over pages during recognition and over questions during grading, with nothing shown until one unit has finished; the batch estimate multiplies the mean duration of completed scripts by the number still queued. Being empirical, the estimator self-corrects when a failover changes the per-question cost part-way through a batch.

IV. EVALUATION

A. What Is and Is Not Measured

The experiments quantify three deterministic properties measurable without authentic student data: whether the segmenter attributes answers to the right question under recognition noise, whether the grading layer recovers or safely rejects malformed output, and whether retrieval stays inside the course being graded. These are the properties on which a mark's attributability depends and they reproduce exactly. They are not a substitute for a marking accuracy study; agreement with human marks on real scripts is not reported here and is described in Section VI. All experiments ran on the released code with fixed seeds and no network access, under Python 3.12 on a Windows workstation.

B. Segmentation Robustness

Two hundred synthetic four-question scripts were generated per condition, each answer body embedding a unique sentinel phrase so that attribution is checked exactly: a question counts as correctly attributed only when its segment contains its own sentinel and no other question's sentinel. Six conditions vary the marker surface form and the surrounding noise, from clean Arabic labels through Roman numerals to labels corrupted by the substitutions recognition produces, with and without injected page banners, rule lines and distractor sentences. The baseline is a digit-only line-initial matcher over the uncleaned transcript, representing a straightforward first implementation. Table III reports recall, the proportion of rubric questions receiving a segment, and attribution, the proportion attributed exactly.

TABLE III. SEGMENTATION RECALL AND ATTRIBUTION, PER CENT, 200 SCRIPTS PER CONDITION

Condition	Proposed R / A	Baseline R / A
Arabic labels, clean	100.0 / 100.0	81.6 / 71.0
Enumerated labels, no letter	100.0 / 100.0	0.0 / 0.0
Roman numerals	100.0 / 100.0	0.0 / 0.0
Roman with substituted glyphs	83.0 / 68.9	0.0 / 0.0
Arabic plus banners and noise	100.0 / 100.0	82.0 / 69.9
Substituted glyphs, banners, distractors	82.6 / 68.0	0.0 / 0.0

The proposed segmenter attributes every question correctly under four of six conditions, including the condition adding banners and rule lines, where the baseline loses about thirty per cent of attributions and pollutes 41.8 per cent of the segments it does produce with banner text, while the proposed cleaning stage leaves no polluted segment in any condition. The baseline fails completely wherever the label is not an Arabic digit preceded by the letter Q, which covers half the conditions. On marker-free scripts neither implementation produced a spurious segment in 200 trials, confirming that the negative lookahead does not trade recall for false positives.

The residual failure under glyph substitution was localised by enumerating thirty marker surface forms individually, of which twenty-six resolved correctly. All four failures share one cause: the Roman numeral lookup uses the raw token, while substitution of vertical bars and lowercase letter forms for the numeral one is normalised only for the pure repeated-one case. A label read as two vertical bars therefore matches a single-character alternative and collapses onto question one, and a mixed form combining a substituted glyph with a genuine numeral resolves to nothing. The remedy is to normalise glyphs before the numeral lookup and treat repeated bars as a repeated numeral, a change confined

to one function; it is not applied in the version measured here, so Table III is a lower bound.

C. Recovery of Malformed Grading Responses

The second experiment replays fourteen response shapes through the parsing and normalisation layer with a fixed two-question rubric and a known correct total; twelve are recoverable in principle and two are not recoverable by design. An outcome is recovered when the total matches exactly, rejected when the parser raises so the router can retry or fail over, and wrong when a mark is produced that does not match. As Table IV shows, all twelve recoverable shapes yielded the correct total and none of the fourteen produced an incorrect mark, so the layer either agrees with ground truth or declines. One qualification belongs in the record: for the syntactically invalid shape the correct total is recovered but per-question attribution is lost and the whole total is attached to the first question. That is visible in the stored rows and in the parse warning attached to the result, and it is why the interface exposes single-question re-grading, but a reviewer looking only at the total would not notice it.

TABLE IV. OUTCOMES FOR FOURTEEN GRADING RESPONSE SHAPES

Response shape	Expected	Outcome
Canonical object with per-question rows	12	recovered
Fenced code block	12	recovered
Envelope with fenced payload	12	recovered
Envelope with zero placeholder total	12	recovered
Echoed prompt before grade object	12	recovered
Flat single mark, no rows	8	recovered
Zero rows, non-zero declared total	9	recovered
Unpadded question numbers	12	recovered
Marks encoded as strings	12	recovered
Declared total omitted	12	recovered
Trailing conversational text	12	recovered
Invalid syntax, fields present	12	recovered
Prose with no machine-readable mark	none	rejected
Empty body	none	rejected

D. Course-Scoped Retrieval

The third experiment builds a throwaway index of twenty-four lecture chunks across three courses and issues four query sets under the deployed policy: in-scope queries paraphrasing rubric questions for the filtered course; cross-course queries paraphrasing questions from a different course while the filter names the graded course; off-syllabus general knowledge queries; and the in-scope queries repeated with the filter disabled, as an ablation. The last three sets should return nothing.

TABLE V. RETRIEVAL OUTCOMES BY QUERY SET

Query set	n	Context	None	Foreign
In-scope, filter on	18	17	1	0
Cross-course, filter on	36	0	36	0
Off-syllabus, filter on	18	0	18	0
In-scope, filter off	18	17	1	0

Retrieval supplied context for seventeen of eighteen in-scope queries at a mean distance of 0.266 against a ceiling of 0.45, and each of those responses drew only on the filtered course. All thirty-six cross-course and all eighteen off-syllabus queries returned no context, so the prompt fell back to rubric-only marking rather than receiving material from the wrong module or from outside the syllabus. One in-scope query fell above the ceiling and was treated as a miss, the expected cost of a conservative threshold: the mark is then made from the rubric and answer alone, a defensible degradation rather than a wrong grounding. The ablation did not show what one might expect, and is reported for that reason. With three semantically well-separated courses, disabling the metadata filter changed nothing, because the distance ceiling alone kept every response inside the correct course. On this corpus the filter is therefore a structural guarantee rather than a measurable gain, and the case it protects against is the one the corpus does not contain, namely overlapping modules sharing vocabulary, where similarity alone cannot identify the right source.

E. Component Tests and an Operational Trace

Four test suites shipped with the service were executed and all passed, covering marker resolution and banner stripping; identifier normalisation, the manual, folder and recognition precedence chain, and all six roster validation categories; unwrapping of envelope-style grading responses including the zero-placeholder case; and six cases covering health probe derivation, probe caching, batch-level engine skipping and cache invalidation on a not-found status.

As an end-to-end illustration, a two-script batch of four questions each was run against the deployed service. Recognition yielded 703 and 300 characters for the two pages of the first script and 809 and 711 for the second; the segmenter found four segments in each script and every question was marked from its own segment. The primary engine passed the health probe but returned a server error on all three grading attempts, whereupon the router marked it unavailable, skipped it for the remaining questions and completed all eight question-level gradings on the fallback engine. The batch finished without operator intervention and the engine used was recorded per question. This is a single observation rather than a performance measurement, but it exercises exactly the path the routing rules exist for and shows why a health probe alone is insufficient: a reachable endpoint is not a working one, so the router must also react to failures observed during grading.

V. DISCUSSION AND THREATS TO VALIDITY

The results support a narrow but useful claim. The parts of the pipeline that decide whether a mark is attributable at all, namely segmentation, response parsing and retrieval scoping, behave predictably and fail in the safe direction: an unattributable answer becomes a visible zero, an unparseable response becomes a retry or a failover, and an ungrounded question becomes rubric-only marking. None of the fourteen malformed responses and none of the fifty-four out-of-scope queries produced a plausible-looking but unfounded result. For a system whose output will be defended to a student, that asymmetry matters more than a marginal gain in average accuracy. The results equally show where the design is weakest: segmentation, not marking, is the current bottleneck on adversarial recognition output, and because attribution errors propagate silently into per-question marks, the one-

function repair identified in Section IV-C is the highest-value change available.

Several threats to validity should be stated plainly. All grading data used so far is synthetic, so nothing here establishes that the marks agree with a human marker; the segmentation and retrieval corpora are author-generated, and although the corruption patterns were taken from observed recognition output, their frequency in real scripts is unknown. The pipeline depends on a third-party recognition service, which constrains offline operation and the handling of student work under data-protection obligations. Both recognition and grading were exercised in English only, and the operational trace covers two scripts, supporting no claim about throughput at cohort scale.

VI. Conclusion and Future Work

This paper described a handwritten grading subsystem that turns photographed answer scripts into per-question, rubric-aligned marks, and reported the decisions that make those marks attributable: a conditioning cascade preserving pen strokes, an escalating recognition policy with an explicit usability test, a recognition-tolerant marker grammar with a conservative attribution rule, retrieval hard-scoped to one course, one model call per rubric question behind a health-gated dual-engine router, and a normalisation layer that recomputes totals rather than trusting them. Experiments showed exact attribution under four of six degradation conditions against a baseline failing entirely on three, recovery of twelve of fourteen malformed responses with no incorrect mark produced, and rejection of all fifty-four out-of-scope retrieval queries with no cross-course leakage.

Four lines of work follow. First, the validation study these results cannot substitute for: with institutional ethical clearance, a corpus of authentic scripts marked independently by two human markers, reported as exact agreement and quadratic weighted kappa against both markers and against their mutual disagreement, so the system is compared to the spread between markers rather than to a single gold mark. Second, replacing the external recognition service with an on-premise transformer-based recogniser [8], removing the network dependency and keeping student work inside institutional infrastructure. Third, replacing in-process background execution with a durable distributed queue so a batch survives a restart, and re-measuring throughput once scripts can be graded in parallel. Fourth, routing low-confidence questions to human review using the diagnostics already stored per mark, namely the segmentation source, the engine used, whether lecture context was found and any parse warning, so the system's own uncertainty decides what a marker must look at.

Acknowledgment

The authors thank the Faculty of Computing, Sri Lanka Institute of Information Technology, for the supervision and infrastructure provided for this research project.

References

[1] S. Burrows, I. Gurevych, and B. Stein, "The eras and trends of automatic short answer grading," Int. J. Artif. Intell. Educ., vol. 25, no. 1, pp. 60-117, 2015.

[2] D. Ramesh and S. K. Sanampudi, "An automated essay scoring systems: a systematic literature review," Artif. Intell. Rev., vol. 55, no. 3, pp. 2495-2527, 2022.

[3] S. Haller, A. Aldea, C. Seifert, and N. Strisciuglio, "Survey on automated short answer grading with deep learning: from word embeddings to transformers," 2022, arXiv:2204.03503.

[4] G. Kortemeyer, "Performance of the pre-trained large language model GPT-4 on automated short answer grading," Discover Artif. Intell., vol. 4, art. 47, 2024.

[5] I. Chamieh, T. Zesch, and K. Giebermann, "LLMs in short answer scoring: limitations and promise of zero-shot and few-shot approaches," in Proc. 19th Workshop Innovative Use of NLP for Building Educational Applications (BEA), Mexico City, Mexico, 2024, pp. 309-315.

[6] L. Zheng et al., "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in Advances in Neural Information Processing Systems, vol. 36, 2023.

[7] U.-V. Marti and H. Bunke, "The IAM-database: an English sentence database for offline handwriting recognition," Int. J. Document Anal. Recognit., vol. 5, no. 1, pp. 39-46, 2002.

[8] M. Li et al., "TrOCR: transformer-based optical character recognition with pre-trained models," in Proc. 37th AAAI Conf. Artificial Intelligence, Washington, DC, USA, 2023, pp. 13094-13102.

[9] R. Smith, "An overview of the Tesseract OCR engine," in Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR), Curitiba, Brazil, 2007, pp. 629-633.

[10] N. Otsu, "A threshold selection method from gray-level histograms," IEEE Trans. Syst., Man, Cybern., vol. 9, no. 1, pp. 62-66, 1979.

[11] R. O. Duda and P. E. Hart, "Use of the Hough transformation to detect lines and curves in pictures," Commun. ACM, vol. 15, no. 1, pp. 11-15, 1972.

[12] J. Canny, "A computational approach to edge detection," IEEE Trans. Pattern Anal. Mach. Intell., vol. PAMI-8, no. 6, pp. 679-698, 1986.

[13] A. Buades, B. Coll, and J.-M. Morel, "A non-local algorithm for image denoising," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), San Diego, CA, USA, 2005, pp. 60-65.

[14] G. Bradski, "The OpenCV library," Dr. Dobb's J. Softw. Tools, vol. 25, no. 11, pp. 120-125, 2000.

[15] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459-9474.

[16] Y. Gao et al., "Retrieval-augmented generation for large language models: a survey," 2023, arXiv:2312.10997.

[17] N. Reimers and I. Gurevych, "Sentence-BERT: sentence embeddings using Siamese BERT-networks," in Proc. Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP), Hong Kong, China, 2019, pp. 3982-3992.

[18] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: deep self-attention distillation for task-agnostic compression of pre-trained transformers," in Advances in Neural Information Processing Systems, vol. 33, 2020.

[19] E. J. Hu et al., "LoRA: low-rank adaptation of large language models," in Proc. 10th Int. Conf. Learning Representations (ICLR), 2022.

[20] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: efficient finetuning of quantized LLMs," in Advances in Neural Information Processing Systems, vol. 36, 2023.

[21] Llama Team, AI at Meta, "The Llama 3 herd of models," 2024, arXiv:2407.21783.

[22] B. Atil et al., "LLM stability: a detailed analysis with some surprises," 2024, arXiv:2408.04667.

[23] O. Zawacki-Richter, V. I. Marin, M. Bond, and F. Gouverneur, "Systematic review of research on artificial intelligence applications in higher education - where are the educators?," Int. J. Educ. Technol. Higher Educ., vol. 16, art. 39, 2019.

[24] A. Mizumoto and M. Eguchi, "Exploring the potential of using an AI language model for automated essay scoring," Res. Methods Appl. Linguist., vol. 2, no. 2, art. 100050, 2023.

[25] "Chroma: the open-source AI application database." [Online]. Available: https://www.trychroma.com

[26] "OCR API," OCR.space. [Online]. Available: https://ocr.space/ocrapi

[27] "Unsloth: finetune LLMs faster with less memory," GitHub repository. [Online]. Available: https://github.com/unslothai/unsloth